RTL and SystemVerilog Refresher

Fundamentals, memory, datapaths, control, and verification

Understand the hardware. Name the state. Trace the clock edge.

This guide is for returning to RTL after time away. Start with the language and clocked-hardware model. Then review the reusable blocks, memory interfaces, and design tradeoffs. The tiny-tpu project supplies concrete connections; the examples teach general design rules. The final sections give a small cocotb workflow and practice tasks for a 90-minute exercise.

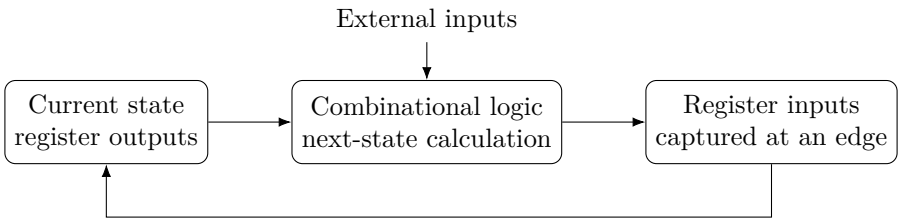
For each topic, recover the purpose before the syntax. Explain why the update works, write the block, then change one assumption. Runnable examples are in `examples/`; their tests check cycle behavior and boundary cases.

Contents

1 Think in hardware and cycles **2**
2 The SystemVerilog syntax you need **2**
3 Combinational logic and clocked state **5**
4 FSMs: control the progress of one transaction **7**
5 Ready/valid: preserve work under backpressure **8**
6 FIFO: derive the state from occupancy **9**
7 Memory: separate the interface from the storage **11**
8 DRAM: enough architecture to reason about a controller **13**
9 Arithmetic, MACs, and fixed-point values **14**
10 Pipelines: move meaning with the data **15**
11 Timing, reset, and clock-domain boundaries **16**
12 Architecture: reuse, locality, and configurable datapaths **17**
13 Cocotb: drive, predict, and check **19**
14 Refresh the tiny-tpu project efficiently **21**
15 Use the refresher in a 90-minute exercise **23**
16 Sources and runnable material **24**

1 Think in hardware and cycles

The goal of RTL is to describe state and the logic that updates it. Registers preserve bits across cycles. Combinational logic computes outputs from current inputs and register values. Modules operate concurrently; source-file order does not schedule hardware modules.



The feedback path passes through registers. A combinational feedback loop usually has no well-defined settling behavior for ordinary RTL design.

Before coding, define the contract.

1. What is transferred, and how wide is it? Is it signed? Is it an integer or fixed-point value?
2. Which edge accepts an input? What makes the input valid? Can either side stall?
3. What latency and sustained throughput are required? Can multiple operations be in flight?
4. What does reset clear? What happens on simultaneous operations or a boundary condition?

For example: "This block stores one byte. It accepts a byte when valid and ready are high at a rising edge. It preserves that byte until the receiver accepts it. Reset discards any pending byte." That contract determines the register, valid bit, and enable equation in §5.

Separate elaboration, synthesis, and simulation. Parameters and generate loops build a fixed circuit before simulation runs. Synthesis maps supported RTL into gates, registers, and available memory resources. Simulation also permits software-like testbench features such as delays and file access; those features do not automatically describe implementable hardware.

For a new design. Write one sentence for each stored fact and each externally visible timing rule. If you cannot say which bits must survive to the next cycle, sketch the datapath before adding an FSM.

2 The SystemVerilog syntax you need

2.1 Modules, ports, signals, and connections

A module is a hardware boundary. Ports connect it to other hardware. Declare vector widths explicitly and use named connections so that port order cannot silently change the wiring.

```
logic [7:0] a, b;
logic [8:0] sum;
assign sum = {1'b0, a} + {1'b0, b};

counter #(.LIMIT(10)) u_counter (
    .clk(clk), .rst(rst), .en(en), .count(count), .tick(tick)
);
```

`logic` is a four-state variable type, not an instruction to build a flip-flop. Its assignment context determines whether it represents combinational logic or registered state. `wire` is a net type used for connections

and continuous drivers. Legacy `reg` denotes a procedural variable; it does not necessarily mean a hardware register either. For ordinary internal signals, give each signal one owner. Do not assign one register from two procedural blocks.

Syntax	Meaning and example
`logic [7:0] data`	One 8-bit packed vector. `data[0]` is its least significant bit.
`logic signed [7:0] delta`	Signed two's-complement value: range −128 through 127. Packed vectors are unsigned unless declared signed.
`logic [7:0] mem [0:15]`	An unpacked array of sixteen 8-bit words. `mem[3][0]` selects one word's low bit.
`logic [1:0][7:0] lanes`	One packed object containing two 8-bit lanes; it can also be treated as a 16-bit vector.
`parameter int W = 8`	Configurable elaboration constant. `localparam` is derived locally and cannot be overridden.
`typedef enum logic [1:0]`	Name a state type and its encodings; use named states instead of unexplained bit patterns.
`typedef struct packed`	Group named fields into one packed vector, useful for an instruction or a data/metadata bundle.

2.2 Literals, slicing, concatenation, and operators

Use sized constants to make bit patterns visible. For example, `8'hFF` is eight one-bits; `8'shFF` is the signed value −1. An unsized decimal literal such as `1` introduces integer sizing and signedness rules. Prefer explicit widths at RTL boundaries.

```
logic [15:0] word;
logic [7:0] byte_value;
assign byte_value = word[15:8];      // constant part select
// word[base +: 8] selects 8 bits starting at base and going up.
// word[base -: 8] selects 8 bits starting at base and going down.
// The base may vary; the selected width must be constant.
assign word = {8'hA5, 8'h3C};        // concatenation: 16'hA53C
// {4{2'b10}} is replication: 8'b10101010.
// '0 fills the destination width with zeros; '1 fills it with ones.
```

This is a syntax fragment: the output slice follows the continuously driven word, which is a constant in this example. Concatenation joins bit fields; replication repeats a pattern.

Operator	What to recall
`+ - * /`	Arithmetic. Constant power-of-two operations can simplify; general division can be expensive hardware.
`& | ^ ~`	Bitwise AND, OR, XOR, NOT. `4'b1010 & 4'b1100` gives `4'b1000`.
`&& || !`	Logical tests. `4'b0010 && 4'b0100` is true even though their bitwise AND is zero.
`&data`, `|data`, `^data`	Reduce all bits to one bit: all-set, any-set, or parity.
`== !=`, `< <= > >=`	Value comparisons. Signedness of the expression matters. In an expression, `<=` means less-than-or-equal.

Operator	What to recall
`=== !==`	Simulation case equality: X and Z are compared as symbols. Useful in tests, not a substitute for an RTL unknown-value policy.
`« » »>`	Left shift, zero-fill right shift, and arithmetic right shift. `»>` sign-extends only when its left operand is signed.
`condition ? a : b`	A mux. Keep branch widths and signedness consistent.

`X` means unknown in four-state simulation; `Z` means high impedance. An unknown comparison can produce X; an `if` does not treat X as definitely true. Avoid `casex` for normal control logic because it can hide real unknowns.

2.3 Widths and signedness are part of the arithmetic

The goal is to preserve the intended numeric range, not just to silence a warning. Two 8-bit unsigned inputs can sum to 510, so the full sum needs 9 bits. Widen operands explicitly when the intermediate result must be preserved.

```
logic [7:0] a, b;
logic [8:0] full_sum;
assign full_sum = {1'b0, a} + {1'b0, b};
// a=200, b=100: full_sum=300; truncating to 8 bits gives 44.

logic signed [7:0] x, y;
logic signed [8:0] signed_sum;
assign signed_sum = $signed({x[7], x}) + $signed({y[7], y});
// x=-100, y=-100: signed_sum=-200.
```

SystemVerilog uses context-dependent expression widths; the destination can affect some arithmetic expressions. Do not assume that every operation has either unlimited width or always the operand width. Explicit extension makes the intended intermediate width visible. `$signed(value)` changes interpretation, not the bit count or binary-point position. A part select and a concatenation are unsigned unless cast or assigned into a suitable signed context. For example, casting `8'hFF` to signed gives -1; zero-extending it to `16'h00FF` gives 255.

2.4 Loops build repeated work or repeated hardware

A combinational loop with a fixed bound describes repeated logic, not several clock cycles. A loop inside a clocked block still executes for one clock event. Use a counter and state when work must be spread across cycles.

```
`timescale 1ns/1ps
module lane_adders #(
    parameter int LANES = 2, W = 8
) (
    input  logic [LANES-1:0][W-1:0] a, b,
    output logic [LANES-1:0][W:0] y
);
    for (genvar i = 0; i < LANES; i++) begin : lane
        assign y[i] = {1'b0, a[i]} + {1'b0, b[i]};
    end
endmodule
```

The generate loop creates `LANES` adders. `genvar` is an elaboration index. For a small helper calculation, a synthesizable `function automatic` can express combinational logic without copied code. A function does not insert a cycle or create a background software task.

Parameter edge cases. An address for `DEPTH` entries needs `$clog2(DEPTH)` bits, but `$clog2(1)` is zero. Guard it with a minimum width of one. A count from 0 through `DEPTH` needs `$clog2(DEPTH+1)` bits. These are different ranges.

For a new design. Before choosing a cast or slice, state the source range, required intermediate range, and destination range. Before writing a loop, decide whether the goal is parallel hardware or one operation reused across cycles.

3 Combinational logic and clocked state

3.1 Combinational logic must decide every output

Use blocking assignments in `always_comb`. Set defaults, then override them. An unassigned output must retain its previous value, which can infer a latch.

```
always_comb begin
    result = '0;
    valid = 1'b0;
    if (enable) begin
        result = a + b;
        valid = 1'b1;
    end
end
```

The hardware is a mux and an adder. Assigning zero on the disabled path is a deliberate behavior. If the requirement is to preserve the previous value, express an enabled register in `always_ff` instead. For a state decoder, use a default next state and a normal `case` with a defined fallback. `unique case` expresses a checking/optimization intent; it does not replace complete assignments or correct encodings.

3.2 At an edge, every register reads the old state

Use nonblocking assignments for register updates. Right-hand sides are evaluated using the state visible when the block executes; the scheduled updates become visible later in that simulation time step.

```
always_ff @(posedge clk) begin
    a <= b;
    b <= a;
end
```

If the old values were `a=1, b=2`, the new values are `a=2, b=1`. This describes two registers exchanging values. It does not perform a sequential software copy. With `a = b; b = a;` in one procedural execution, the second statement would read the changed `a`.

```
always_ff @(posedge clk) begin
    if (rst) q <= '0;
    else if (en) q <= d;
end
```

There is no assignment when `en` is false, so the flip-flop holds its value. That is intentional state retention, not a latch. Omitting a register from reset can also be intentional if a reset valid bit makes its data irrelevant. Within one block, later executed nonblocking assignments to the same variable override earlier ones for that edge. Prefer one clear priority chain; never depend on scheduling order between separate blocks.

3.3 A counter makes the boundary explicit

The counter runs from zero to `LIMIT-1`. An enabled terminal edge wraps it and produces one cycle of `tick`. Reset has priority over enable.

```
`timescale 1ns/1ps
module counter #(
    parameter int LIMIT = 10,
    localparam int CW = (LIMIT <= 1) ? 1 : $clog2(LIMIT)
) (
    input  logic clk, rst, en,
    output logic [CW-1:0] count,
    output logic tick
);
    // Contract: LIMIT >= 1. Reset is synchronous and active high.
    always_ff @(posedge clk) begin
        if (rst) begin
            count <= '0;
            tick <= 1'b0;
        end else begin
            tick <= 1'b0;
            if (en) begin
                if (count == CW'(LIMIT-1)) begin
                    count <= '0;
                    tick <= 1'b1;
                end else begin
                    count <= count + 1'b1;
                end
            end
        end
    end
endmodule
```

`CW'(LIMIT-1)` is a sized cast to `CW` bits. For `LIMIT=3` and enable continuously high, post-edge counts are `1,2,0,1,2,0`; ticks are `0,0,1,0,0,1`. For `LIMIT=1`, every enabled edge is terminal. The guarded width

keeps that configuration legal.

For a new design. Write a table of old state, input conditions, and new state. Resolve priority explicitly when reset, load, clear, and increment can coincide.

4 FSMs: control the progress of one transaction

An FSM is useful when the same datapath does different work on different cycles. The state should describe the transaction's progress, not merely label lines of code.

Consider a block that sums exactly `N` incoming bytes. A start command captures `N`; input bubbles must not consume a byte. Use three states: IDLE accepts a command, RUN accepts data, DONE reports completion for one cycle.

State	Event	Action / next state
IDLE	Start with $N > 0$	Clear sum, capture count, enter RUN.
IDLE	Start with $N = 0$	Clear sum and enter DONE.
RUN	Valid input accepted	Add data and decrement remaining count; enter DONE on the last accepted item.
RUN	No accepted input	Hold sum and remaining count.
DONE	Next edge	Return to IDLE.

The remaining count means "items still to be accepted", so its old value is 1 on the final transfer. Do not decrement merely because a cycle passed.

```
typedef enum logic [1:0] {IDLE, RUN, DONE} state_t;
state_t state, next_state;
logic [4:0] left;
assign in_ready = (state == RUN);
assign busy = (state != IDLE);
assign done = (state == DONE);
always_comb begin
    next_state = state;
    case (state)
        IDLE: if (start) begin
            if (command_count == 0) next_state = DONE;
            else next_state = RUN;
        end
        RUN: if (in_valid && in_ready && left == 1)
            next_state = DONE;
        DONE: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end
```

The next-state block computes only a decision. The sequential block owns the stored state and datapath registers.

```
    always_ff @(posedge clk) begin
        if (rst) begin
            state <= IDLE;
            left <= '0;
            sum <= '0;
        end else begin
            state <= next_state;
            if (state == IDLE && start) begin
                left <= command_count;
                sum <= '0;
            end else if (state == RUN && in_valid && in_ready) begin
                sum <= sum + {8'b0, in_data};
                left <= left - 1'b1;
            end
        end
    end
```

The full file supplies the ports and declarations. `in_ready` is true only in RUN; `done` is true in DONE. The final data edge updates both `sum` and the state, so the completed sum is available while `done` is high. Starts while busy are ignored by this contract. A held start can launch another command once IDLE returns; use a pulse or a proper command handshake.

For a new design. Separate command acceptance, request issue, response wait, and completion when their events occur on different cycles. Capture command fields once instead of letting a busy transaction follow live input wires.

5 Ready/valid: preserve work under backpressure

The goal is to let either side pause without losing, duplicating, or changing an item. Outside reset, a transfer occurs at a rising edge when `valid && ready` is true. The sender owns valid and data; the receiver owns ready. Once valid is asserted, the sender keeps valid and the same data until acceptance. It must not wait for ready before making a pending item valid.

5.1 Derive a one-entry elastic register

Store the data and a bit saying whether the slot is occupied. The slot can accept input if it is empty or if the receiver will remove the current item at this edge. Therefore `in_ready = !out_valid || out_ready`.

```
`timescale 1ns/1ps
module rv_register #(
    parameter int W = 8
) (
    input  logic clk, rst,
    input  logic in_valid,
    output logic in_ready,
    input  logic [W-1:0] in_data,
    output logic out_valid,
    input  logic out_ready,
    output logic [W-1:0] out_data
);
    assign in_ready = !out_valid || out_ready;
    always_ff @(posedge clk) begin
        if (rst) begin
            out_valid <= 1'b0;
        end else if (in_ready) begin
            out_valid <= in_valid;
            if (in_valid)
                out_data <= in_data;
        end
    end
endmodule
```

When stalled with a valid output, `in_ready` is false and the block changes nothing. When an old item leaves, a new item can replace it at the same edge. When no new item arrives and the old one leaves, the valid bit clears. The data register is not reset because its value is meaningful only when valid is high.

Pre-edge condition	Result after the edge
Empty, new valid item A	A occupies the slot. It was not also transferred to the receiver through an empty bypass.
Occupied by A, receiver stalled	A and output valid hold. No input is accepted.
Occupied by A, receiver ready, new B valid	A transfers out; B replaces it. Occupancy stays one.
Occupied by A, receiver ready, no new input	A transfers out; the slot becomes empty.

This block can sustain one item per cycle after filling. It adds a register boundary, not an empty-path combinational bypass. Several such stages can form a long combinational ready path. Registering ready alone is unsafe if an already accepted item then has nowhere to go. Break that path with a correctly sized skid/elastic buffer, or use credits that reserve storage.

For a new design. Define transfers from pre-edge values. Attach all metadata to the same acceptance and stall rules as its data. Before adding a register to a control path, account for every item that may still arrive before the sender observes the stall.

6 FIFO: derive the state from occupancy

A FIFO absorbs a temporary difference between production and consumption rates while preserving order. Store the words, a read pointer, a write pointer, and the number of occupied slots. Define `push` and `pop` from accepted transfers, not from requests alone.

Push	Pop	Occupancy change
0	0	Hold.
1	0	Add one.
0	1	Subtract one.
1	1	Hold; both pointers advance.

The invariant is $0 \leq \text{count} \leq \text{DEPTH}$. Empty means count is zero; full means count equals depth. A full FIFO can accept a replacement when the old head will leave at the same edge. An empty FIFO cannot pop in this design, even if a new input arrives at that edge.

```
`timescale 1ns/1ps
module sync_fifo #(
    parameter int W = 8, DEPTH = 4,
    localparam int AW = (DEPTH <= 1) ? 1 : $clog2(DEPTH),
    localparam int CW = $clog2(DEPTH+1)
) (
    input  logic clk, rst,
    input  logic in_valid,
    output logic in_ready,
    input  logic [W-1:0] in_data,
    output logic out_valid,
    input  logic out_ready,
    output logic [W-1:0] out_data
);
    // Contract: DEPTH >= 1; combinational head read, no empty bypass.
    logic [W-1:0] mem [0:DEPTH-1];
    logic [AW-1:0] rd_ptr, wr_ptr;
    logic [CW-1:0] count;
    logic push, pop;
    assign out_valid = (count != 0);
    assign out_data = mem[rd_ptr];
    assign pop = out_valid && out_ready;
    assign in_ready = (count < CW'(DEPTH)) || pop;
    assign push = in_valid && in_ready;
    always_ff @(posedge clk) begin
        if (rst) begin
            rd_ptr <= '0;
            wr_ptr <= '0;
            count <= '0;
        end else begin
            if (push) begin
                mem[wr_ptr] <= in_data;
                wr_ptr <= (wr_ptr == AW'(DEPTH-1)) ? '0 : wr_ptr + 1'b1;
            end
            if (pop)
                rd_ptr <= (rd_ptr == AW'(DEPTH-1)) ? '0 : rd_ptr + 1'b1;
            case ({push, pop})
                2'b10: count <= count + 1'b1;
                2'b01: count <= count - 1'b1;
                default: ;
            endcase
        end
    end
endmodule
```

The casts `AW'(...)` and `CW'(...)` size constant comparisons explicitly. The pointer width represents addresses; the count width must also represent DEPTH itself. Explicitly wrapping at `DEPTH-1` supports depth 3 as well as powers of two. At depth 1, simultaneous push/pop transfers the old word and replaces it with the new word. Reset discards all entries by clearing pointers and count; it need not clear the data array.

The head read is combinational. This is useful for a small register-based FIFO, but it does not match a synchronous-read SRAM directly. With synchronous SRAM, request the head in advance and hold the returned word in an output register. Track both stored words and any outstanding read so that prefetch does not duplicate or skip a word.

For a new design. Choose the memory interface before choosing FIFO control. Ask whether an input may bypass an empty FIFO, whether a full FIFO may accept during a pop, and whether ready can depend combinationally on downstream ready. Those choices determine the boundary logic.

7 Memory: separate the interface from the storage

7.1 Registers, SRAM, and DRAM

An RTL array describes indexed storage. It does not, by itself, promise a physical memory macro. The access pattern, reset behavior, synthesis tool, and target technology determine the implementation.

Storage	Useful mental model	Design consequence
Registers	Individually accessible state bits with surrounding mux logic.	Flexible ports and reset, but large arrays and wide selection muxes cost area and timing.
SRAM	Dense static storage with a fixed set of ports and a defined timing contract.	Match port count, access latency, masks, and collision behavior. An ASIC macro is an explicit implementation resource.
DRAM	Dense dynamic storage, usually reached through a controller and PHY.	Refresh, row operations, scheduling, and bursts affect service time. Buffer requests and responses.

A **1RW** memory has one port that reads or writes. A **1R1W** memory has a read port and a write port. A **2RW** memory has two read/write ports, with specified restrictions on collisions. "Dual port" alone does not define which operations may occur together. Two addresses in one cycle require two available accesses, separate banks, replication, or a schedule that serializes them.

7.2 A synchronous-read RAM model

The request address and enable are sampled at a rising edge. Read data and its valid bit update after that edge. The model below accepts one read and one write per cycle, with valid addresses in `0..DEPTH-1`. It specifies old data when the same address is read and written at the same edge.

```
`timescale 1ns/1ps
module sync_ram #(
    parameter int W = 8, DEPTH = 8,
    localparam int AW = (DEPTH <= 1) ? 1 : $clog2(DEPTH)
) (
    input  logic clk, rst,
    input  logic wr_en, rd_en,
    input  logic [AW-1:0] wr_addr, rd_addr,
    input  logic [W-1:0] wr_data,
    output logic rd_valid,
    output logic [W-1:0] rd_data
);
    // Valid addresses only. This model specifies old data on a collision.
    logic [W-1:0] mem [0:DEPTH-1];
    always_ff @(posedge clk) begin
        if (rst) begin
            rd_valid <= 1'b0;
        end else begin
            rd_valid <= rd_en;
            if (rd_en)
                rd_data <= mem[rd_addr];
            if (wr_en)
                mem[wr_addr] <= wr_data;
        end
    end
endmodule
```

Both right-hand sides see the pre-edge memory contents. A later write statement does not make this a new-data read. A physical macro may instead return new data, preserve its old output, or leave that collision undefined. Match the macro's specification, or add forwarding/arbitration to provide the external behavior you need.

Time	RAM	A separate downstream register
Before edge E0	Read enable and address A are stable.	Sees the previous response.
At E0	Samples the request.	Still samples the previous response, not A's new data.
After E0 settles	`rd_valid=1`; `rd_data=mem[A]`.	Combinational consumers can now use A's response.
At E1	May accept the next request.	Can capture A's response.

Calling this a one-cycle read is common, but an edge table removes ambiguity. A controller that issues a request and consumes its response in the same sequential step will use stale data.

7.3 Reset, ports, addresses, and inference

Reset valid bits and ownership state first. A large loop that clears every word in one clock asks for hardware that writes every word at once; an ordinary SRAM cannot do that. If contents must be initialized, use an initialization sequence, valid tags, or a supported memory initialization mechanism. Unwritten locations in this model are unspecified; do not test them as zero.

A byte address and a word index are different. In a byte-addressed system with aligned 32-bit words, address 12 selects word index 3; the two low address bits select a byte within the word. Four byte enables

permit partial writes without replacing the other bytes, if the memory supports them. For a 16-bit word, two byte lanes are enough. Check alignment and the interface contract before shifting an address.

For FPGA RAM inference, use the vendor's supported template and confirm the synthesis report. For an ASIC, a macro wrapper must match the macro's functional model, timing view, physical view, and pin polarities. A declaration with three independent reads does not become a two-port SRAM merely because it is called a buffer.

For a new design. List every access that must occur in one cycle. Then assign each access to a real port or bank. Treat latency, read/write collisions, byte masks, and reset as interface requirements, not implementation details to discover later.

8 DRAM: enough architecture to reason about a controller

DRAM stores charge that must be refreshed. Think of a bank as having rows and a currently open row. An ACTIVATE opens a row; READ or WRITE commands move column data in bursts; PRECHARGE prepares the bank to open another row. These commands have timing constraints. A row hit can avoid closing and reopening a row, but it still obeys column and bus timing.

Term	What it constrains
t_{RCD}	Delay from ACTIVATE to a permitted READ/WRITE.
t_{RP}	Time after PRECHARGE before a subsequent ACTIVATE.
t_{RAS}	Minimum time a row stays active before it can be precharged.
t_{RFC}	Refresh recovery time, according to the device's refresh mode.

For example, a conflicting row request may require PRECHARGE, a wait, ACTIVATE, another wait, then a column command and data transfer. Other constraints also apply: command spacing, write recovery, read/write turnaround, and interactions across banks. Use the selected device/controller specification for actual values; these four names are not a complete scheduler. An RTL client normally talks to a controller's request/response interface rather than driving DRAM pins directly.

Latency and bandwidth solve different problems. Latency is how long one request takes. Bandwidth is how much data can be sustained over time. If a downstream interface can return 16 bytes per cycle but a request takes 20 cycles to return, roughly $16 \times 20 = 320$ bytes of useful work must be in flight to cover that delay at full rate. That is twenty 16-byte responses, assuming enough independent requests and no other bottleneck. One outstanding request followed by a wait cannot achieve that throughput.

Use bursts for contiguous tiles, buffer responses, and reserve destination capacity before issuing work that cannot be stopped. Track accepted requests separately from received responses. A burst counter advances on accepted beats, not merely on elapsed clocks. If response order is not guaranteed, carry transaction IDs and reorder or route results correctly.

For a new design. Ask what limits useful throughput: request issue rate, outstanding capacity, row/bank conflicts, data-bus use, or consumption rate. Buffering can hide latency; it cannot create memory bandwidth.

9 Arithmetic, MACs, and fixed-point values

9.1 Choose the numeric range before the width

Multiplying two signed W-bit integers requires up to 2W bits for the full product. Adding up to K such products needs a conservative extra $\lceil \log_2 K \rceil$ bits, assuming no larger initial bias or accumulator value. For sixteen signed 8-bit products, a 20-bit accumulator is a safe choice: $16 + 4 = 20$. The width is a range budget, not a rule that makes arbitrary later additions safe.

```
`timescale 1ns/1ps
module mac #(
    parameter int W = 8, AW = 20
) (
    input  logic clk, rst, in_valid,
    input  logic signed [W-1:0] x, weight,
    input  logic signed [AW-1:0] acc_in,
    output logic out_valid,
    output logic signed [AW-1:0] result
);
    // Contract: AW >= 2*W. Arithmetic wraps if AW bits overflow.
    logic signed [2*W-1:0] product;
    logic signed [AW-1:0] product_extended;
    assign product = x * weight;
    assign product_extended = product;
    always_ff @(posedge clk) begin
        if (rst) begin
            out_valid <= 1'b0;
            result <= '0;
        end else begin
            out_valid <= in_valid;
            if (in_valid)
                result <= acc_in + product_extended;
        end
    end
endmodule
```

This block computes one signed integer MAC result at an accepting edge and registers its valid flag with that result. It has no ready input; it accepts every valid cycle and cannot absorb downstream backpressure. The separate signed `product_extended` makes sign extension into the accumulator width explicit. Overflow wraps here. Saturation would require computing a wider result and clamping it before narrowing.

9.2 A binary point is an interpretation of stored bits

For a signed 16-bit value with 8 fractional bits, the real value is stored integer divided by 256. Its range is -128 through $127 + 255/256$. This guide uses "16 bits, F=8" to avoid differing conventions for whether Q-format integer bits include the sign.

For example, 1.5 is stored as 384, and -0.5 is stored as -128. Their full product is -49152, now with 16 fractional bits. To return to F=8, shift the wide signed product right by 8: -192, representing -0.75.

```
logic signed [15:0] a, b;
logic signed [31:0] product, scaled;
assign product = a * b;        // F=16 when inputs have F=8
assign scaled = product >>> 8; // F=8, still wide
// Check scaled against the 16-bit limits before narrowing.
```

Arithmetic right shift rounds negative nonmultiples toward negative infinity: $-3 >>> 1 = -2$. It is not truncation toward zero. State whether you want floor, nearest with a defined tie rule, or another policy. Do not add a universal "rounding constant" without checking negative values and tie handling.

Addition requires the same binary-point position. For example, align a bias to the accumulator's scale before adding it. Keeping full products through a dot product and rounding once usually loses less precision than rounding each product, but it costs wider storage and adders. Repeated per-PE requantization can produce a different numerical answer. A sticky overflow bit records that an error occurred; it does not repair the result.

For a new design. Write the signed range and fractional-bit count on every arithmetic boundary. Then trace one positive value, one negative value, and one overflow case through extension, multiplication, rescaling, rounding, and narrowing.

10 Pipelines: move meaning with the data

A pipeline divides a long combinational calculation with registers. **Latency** is the number of cycle intervals from acceptance to an available result. **Initiation interval** is the spacing between accepted operations. A three-stage pipeline can have latency three and accept a new item every cycle. Use an edge table to specify whether you count the input-capture edge as stage one.

Suppose stage 1 registers a product and stage 2 adds a bias. Stage 2 reads the previous product, so it must also read the previous item's bias and valid bit.

```
// product1 is signed and wide enough; bias1/result use accumulator width.
always_ff @(posedge clk) begin
    if (rst) begin
        valid1 <= 1'b0;
        valid2 <= 1'b0;
    end else begin
        valid1 <= in_valid;
        valid2 <= valid1;
        if (in_valid) begin
            product1 <= x * weight;
            bias1 <= bias;
        end
        if (valid1)
            result <= product1 + bias1;
    end
end
```

At E0, item A enters stage 1. At E1, A's result enters stage 2 while B may enter stage 1. After E1, `valid2` describes A's result. A bubble at the input must produce a bubble at the corresponding output slot. Tags, addresses, destination IDs, last-item flags, and operation modes need the same alignment as their data.

The example has no stalls. If a stage stalls, its data, valid, and metadata must all hold. Either stall the whole pipeline under one enable, or give each stage its own elastic acceptance rule. A freely running valid shift register beside a stalled datapath breaks alignment. Changing an output mux mode while an older

item is in flight can send that item down the wrong path; carry the mode or drain before changing it.

A pipelined multiplier in a feedback accumulator needs extra thought: the accumulator depends on an earlier result. Track which partial sum belongs to each product, interleave independent accumulations, or accept a larger initiation interval. Adding registers changes the schedule; it does not automatically preserve a recurrence.

10.1 Double buffering and a weight commit edge

Keep an active bank for computation and a shadow bank for loading the next job. Ownership prevents the loader from changing values in use. Swap ownership only at a defined boundary.

```
`timescale 1ns/1ps
module weight_bank #(
    parameter int W = 8
) (
    input  logic clk, rst, load, swap,
    input  logic signed [W-1:0] weight_in,
    output logic signed [W-1:0] active, shadow,
    output logic signed [W-1:0] selected
);
    // Explicit bypass if a datapath must use the shadow on the swap edge.
    assign selected = swap ? shadow : active;
    always_ff @(posedge clk) begin
        if (rst) begin
            active <= '0;
            shadow <= '0;
        end else begin
            if (load) shadow <= weight_in;
            if (swap) active <= shadow;
        end
    end
endmodule
```

With only `active <= shadow`, a separate MAC sampling `active` at the swap edge still uses the old active value. The optional combinational `selected` mux lets that MAC sample the pre-edge shadow value at the swap edge instead. If load and swap are both asserted, active receives the old shadow while shadow receives the new input. The mux does not magically forward that incoming load; that would require another explicit rule.

For a new design. Whenever a register is added, removed, enabled, or bypassed, trace data and its control through two consecutive items and one bubble. For bank swaps, specify exactly which version a computation at the boundary must observe.

11 Timing, reset, and clock-domain boundaries

11.1 What a timing path means

For two registers on the same clock, a simplified setup condition is

$$t_{\text{clk}\to\text{Q}} + t_{\text{comb,max}} + t_{\text{setup}} \le T_{\text{clk}}.$$

Clock skew, uncertainty, and constraints modify the available budget. A deep adder/mux chain can miss setup even when its functional simulation is correct. Reduce logic depth, reduce fanout, change the

arithmetic structure, or insert a pipeline register and update the protocol latency.

Hold checks concern the *minimum* path delay just after the capturing edge. Lowering the clock frequency generally does not fix a hold violation on a same-edge path. Physical implementation tools add delay or change routing as needed under correct constraints. RTL simulation verifies behavior; synthesis and static timing analysis answer different questions.

Use `if (enable)` inside a clocked block for an enable. Do not casually create a clock with `clk & enable`: enable changes can make glitches or unintended edges. ASIC clock gating uses dedicated glitch-safe cells and clock-tree/verification support; FPGA targets provide their own supported clock resources.

11.2 Reset establishes a valid starting protocol state

A synchronous reset takes effect only at the active clock edge. An asynchronous reset can assert without that edge. Asynchronous reset release near an edge can violate recovery/removal timing; synchronize release within each clock domain when using that strategy. Choose reset polarity and style to match the system and library rather than applying one style universally. Reset FSMs, valid bits, pointers, and ownership as needed. Data bits may remain unreset when their validity makes them unobservable. A synchronous reset cannot clear a block whose clock is stopped until that clock runs.

11.3 Crossing a clock boundary changes the problem

A receiving flip-flop can become metastable when an asynchronous signal changes near its sampling edge. Two receiving-domain synchronizer registers reduce the chance that metastability propagates; they do not make the first register immune.

```
// For a suitably stable single-bit level; reset style omitted here.
always_ff @(posedge dst_clk) begin
    sync1 <= async_level;
    sync2 <= sync1;
end
```

This does not guarantee capture of a narrow pulse. Use a pulse-stretch rule, a toggle protocol with a safe event spacing, or a request/acknowledge handshake when every event must arrive. Independently synchronizing the bits of a bus can capture a mixture of two words. For occasional words, hold the bus stable under a handshake. For a stream, use a correctly designed asynchronous FIFO.

An asynchronous FIFO commonly crosses Gray-coded pointers through synchronizers, then derives local full/empty decisions. Gray coding changes one bit per pointer step; it does not remove the need for the proper FIFO equations, synchronizers, physical CDC constraints, and reset protocol. A ready/valid connection alone is not a clock-domain crossing solution.

For a new design. First identify whether two endpoints share a clock. Then choose a transfer protocol that preserves a level, event, word, or stream as required. Do not start with "add two flops" before deciding what must be preserved.

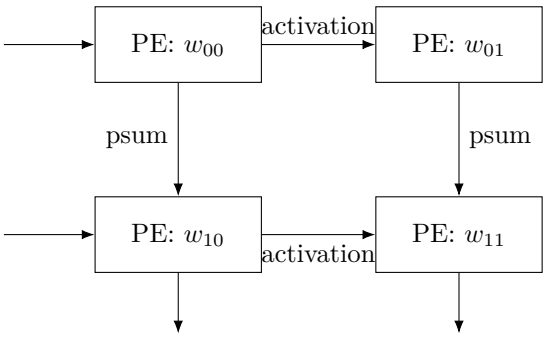
12 Architecture: reuse, locality, and configurable datapaths

12.1 From a MAC to a systolic array

For $C_{ij} = \sum_k A_{ik} B_{kj}$, each output needs repeated multiply-add work. A weight-stationary array keeps a weight in each PE. Activations move horizontally; partial sums move vertically. Registers make each hop

local and give a regular schedule.

In a two-row column, the upper PE can form $a_0 w_0 + p$ at E0. The lower PE can add $a_1 w_1$ to that partial sum at E1. Therefore the matching lower-row activation must be present for E1, not merely arrive at the array at the same time as a_0. Likewise, an activation reaches a later column later. Boundary skew or equivalent internal delay aligns operands for each output. An enable signal cannot correct a value that belongs to the wrong dot product.



A fully occupied $N \times N$ array can perform N^2 MACs per cycle, if each PE supports one MAC per cycle and data arrives correctly. That is peak arithmetic capacity. Small tiles, fill/drain cycles, missing operands, and memory stalls reduce useful utilization. Count operations and transfers for a complete tile rather than quoting the peak alone.

12.2 Choose reuse before increasing parallelism

A scalar engine reuses one MAC over many cycles. Unrolling work creates more MACs and raises operand bandwidth demand. Tiling keeps a useful subset of weights, activations, or partial sums near computation so that external memory traffic is amortized. Choose the resident quantity from the access pattern: reuse weights across examples, activations across outputs, or keep partial sums local to avoid repeated writes.

Banking creates independent accesses when addresses land in different banks. For B banks with simple word interleaving, bank = word address mod B. With four banks, addresses 0,1,2,3 can be read together from four single-read banks; addresses 0,4,8,12 conflict in bank 0. Changing the layout or schedule can resolve the conflict. More compute lanes alone cannot.

Ping-pong tile buffers can overlap loading tile $n + 1$ with computing tile n. Ideal steady-state tile time approaches the larger of load time and compute time instead of their sum, provided the memories, ports, and transfer engines actually permit overlap. Add startup/drain costs and any shared writeback bottleneck.

12.3 A general view of a CGRA

A coarse-grained reconfigurable array connects word-level processing elements with configurable operations and routes. Compared with a fixed systolic path, it can map several datapaths by changing PE functions, routing, and schedules. The design still obeys the same constraints: operand availability, port conflicts, route capacity, register boundaries, and numeric widths.

For example, mapping $(a+b) \times c$ requires an adder PE, a multiplier PE, and delivery of the matching c. If the adder result crosses one register boundary, delay c by the same amount or schedule its arrival later. If two operands need the same single-port source memory at once, add a bank or change the schedule. Keep routing configuration stable for in-flight work, carry configuration identity, or drain before reconfiguration.

Local neighbor routes are simple but constrain mappings; a broad crossbar is flexible but can cost area, wiring, and timing.

For a new design. For any accelerator, trace one output backward: which operands, stored where, arriving on which cycle, through which ports? Then estimate useful operations per byte and ask whether the memory system can feed the chosen parallelism.

13 Cocotb: drive, predict, and check

Cocotb lets Python drive and observe an HDL simulation. The RTL still supplies clocked hardware behavior. Start with one clock, a few inputs, an independent reference model, and assertions. You do not need a large verification framework to test a counter, FIFO, or datapath well.

Operation	Purpose
`@cocotb.test()`	Register an asynchronous Python test.
`dut.signal.value = value`	Schedule an input value. Read with `int(dut.signal.value)` when its bits are known.
`cocotb.start_soon(...)`	Run a clock concurrently with the test: pass `Clock(...).start()`.
`await FallingEdge(dut.clk)`	A convenient point to drive inputs for the next rising edge.
`await RisingEdge(dut.clk)`	Wait for the clock edge; downstream sequential updates may not yet have settled.
`await ReadOnly()`	Observe the settled values for this simulation time step. Do not drive signals in this phase.
`assert actual == expected`	Fail immediately when the observed behavior violates the model.

The examples use cocotb 2.0.0 and `unit="ns"`. Some older project tests use `units="ns"`. The API spelling matters much less than the observation phase.

13.1 Transfers use pre-edge values; state checks use post-edge values

For a ready/valid block, drive at the falling edge, then let combinational outputs settle with `ReadOnly()`. Those values determine the next rising-edge transfers, assuming inputs remain stable until that edge. Update the reference queue from those transfers. Then wait for `RisingEdge()` and `ReadOnly()` to inspect the updated state. Do not decide whether an old word was accepted by looking only at the new output after that edge.

This complete test targets `rv_register.sv`. The sender preserves its pending item until acceptance, and the receiver deliberately stalls.

```python
from collections import deque
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, FallingEdge, RisingEdge, ReadOnly

@cocotb.test()
async def preserve_order_under_stall(dut):
    dut.rst.value = 1
    dut.in_valid.value = 0
    dut.out_ready.value = 0
    dut.in_data.value = 0
    cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())
    await ClockCycles(dut.clk, 2)
    await FallingEdge(dut.clk)
    dut.rst.value = 0
    expected = deque()
    source = iter([11, 22, 33, 44])
    pending = next(source, None)
    for cycle in range(40):
        await FallingEdge(dut.clk)
        dut.in_valid.value = int(pending is not None)
        dut.in_data.value = pending if pending is not None else 0
        dut.out_ready.value = int(cycle > 4 and cycle % 4 != 0)
        await ReadOnly()               # settled, before the accepting edge
        if dut.out_valid.value == 1 and dut.out_ready.value == 1:
            assert expected
            assert int(dut.out_data.value) == expected.popleft()
        if pending is not None and dut.in_ready.value == 1:
            expected.append(pending)
            pending = next(source, None)
        await RisingEdge(dut.clk)
        await ReadOnly()               # settled state after the edge
        if pending is None and not expected:
            return
    assert False, "Timed out waiting for all four bytes"
```

Check the old output before appending the new input. This matches the register's lack of an empty bypass. For an empty queue, newly accepted input must not count as an output at that same edge. The timeout catches missing results; the queue catches wrong values and order. A deeper randomized test should also check that output valid and data remain stable throughout a stall.

13.2 Test the contract's difficult boundaries

Use short directed cases first, then random traffic with a reproducible seed.

Block	Cases that reveal the design decisions
Counter	Disabled at terminal count; limit 1; non-power-of-two limit; reset during counting.
FIFO	Empty pop; fill until full; full replacement; simultaneous push/pop; long stalls; wraparound; depth 1 and depth 3; reset while occupied.
RAM	First valid read after initialization; disabled read; same-address read/write; consecutive requests; reset without clearing stored words.

Block	Cases that reveal the design decisions
MAC	Zero, negative inputs, signed limits, overflow policy, valid bubbles, back-to-back items.
Controller	Zero-length and one-item commands; bubbles before the last item; command changes while busy; reset mid-operation.
Pipeline	Two distinguishable consecutive items; an isolated bubble; changing metadata; a downstream stall if supported.

Python integers have unlimited precision; RTL values do not. Apply the specified masks, signed decoding, rounding, and saturation in the reference model. For example, decode an unsigned W-bit bit pattern u as $u - 2^W$ when its sign bit is set. Use an independently expressed model: a Python queue for a FIFO and ordinary arithmetic for a MAC, rather than copying the RTL pointer equations line for line.

Waveforms help answer a concrete question: "At which first edge did valid, ownership, or data diverge?" Check the first mismatch and its preceding edge before scanning a long trace.

Assertions can also state invariants such as occupancy bounds and stable stalled output. For example, this concurrent SystemVerilog assertion says a stalled word survives to the next sampled edge unless reset intervenes:

```
assert property (@(posedge clk) disable iff (rst)
    out_valid && !out_ready |=> out_valid && $stable(out_data));
```

Simulator support for concurrent assertions varies. The supplied Icarus tests check the small examples through Python; this assertion is an illustrative property, not part of those builds. In formal verification, assumptions constrain legal input behavior; assertions check the DUT's obligations. Overrestricting inputs can make a broken design appear correct.

For a new design. Turn each sentence of the interface contract into a check. Include both correctness of returned values and conservation of accepted work: no unexplained loss, duplication, or reordering.

14 Refresh the tiny-tpu project efficiently

These notes refer to public commit `04ad6923`. The SystemVerilog development tree and the hardened Verilog tree differ. Read one version consistently before comparing variants. This is a source review, not a claim that the complete project has been reverified or physically implemented here.

14.1 Recover the dataflow, then the schedule

In `src/tpu.sv`, the main loop is:

$$\text{unified buffer} \longrightarrow \text{systolic array} \longrightarrow \text{vector unit} \longrightarrow \text{unified buffer}.$$

Host inputs load the buffer and supply controls. This top module does not instantiate the separate instruction decoder. The systolic module explicitly instantiates a 2×2 PE grid; a width parameter elsewhere does not make the entire implementation a generated arbitrary-size array.

Read in this order	Recover this idea
`src/pe.sv`	One PE: active/shadow weights, activation forwarding, partial-sum MAC, valid forwarding, overflow state. Identify which pre-edge values feed each registered output.
`src/systolic.sv`	Horizontal activations, vertical partial sums/weights, propagated switch control, and separate row-start inputs. Trace skew and valid timing.
`src/vpu.sv` and child modules	Bias, activation, loss, and derivative routes. Distinguish combinational bypasses from registered stages before counting latency.
`src/unified_buffer.sv`	Storage, read/write channels, pointer updates, registered outputs, and priority between a new command and ongoing work.
`src/tpu.sv`	Connections and control ownership. Trace one result back to its accepted operands.
`src/control_unit.sv`; `src/fixedpoint.sv`	Instruction field slicing and exact numeric semantics. Compare the selected RTL against encoders/tests.
`test/`; `sva/`	Stimulus, expected results, assertions, and environmental assumptions. Separate a test that merely waits from one that checks an outcome.

14.2 Five project details that generalize

1. A swap changes future register state. In the pinned SystemVerilog PE, active and shadow weights have one sequential owner. A MAC at the edge where `switch_in` is asserted uses the old active weight; the new active value affects subsequent computation. Loading and switching together commits the old shadow value. This is the old-state rule from §10, not a special property of a TPU. Do not import a bypass assumption from another implementation variant.

2. Valid signals carry timing information. The array's two input rows have separate start/valid inputs. A lower row does not necessarily accept the corresponding activation on the upper row's acceptance edge. Trace an activation, its partial sum, and its valid bit through each PE. Give two consecutive dot products different values so that an alignment error cannot hide behind repeated data.

3. A behavioral buffer is not a physical SRAM specification. The unified buffer exposes several logical channels and resets array contents in RTL. Those semantics require a port/reset implementation strategy before mapping to ordinary SRAM macros. Its blocking temporary calculations allow several address calculations within one clocked process, followed by a final nonblocking pointer update. A fresh command has priority over some ongoing-channel updates by assignment order; identify that priority before refactoring. The inspected design does not contain a DRAM controller to study as an implemented subsystem.

4. Numeric behavior belongs to an implementation. The development datapath uses signed 16-bit values with 8 fractional bits and fixed-point helper modules. The hardened tree includes a simplified fixed-point implementation; do not assume identical rounding, saturation, or overflow reporting across variants. Read the selected multiply/add path with the questions in §9.

5. Formats and modes must stay aligned. The pinned `src/control_unit.sv` decodes a 130-bit instruction; the README describes 94 bits and the hardened decoder uses 88 bits. The field map is version-dependent. Follow the selected RTL and matching producer/tests instead of memorizing one stale diagram. Likewise, the VPU has selectable paths with different register boundaries. Hold control stable for the required transaction lifetime or carry matching metadata with in-flight data.

14.3 A small project walkthrough you can do from memory

Choose one output element of a two-row dot product. Write its two weights and two activations as small signed values. First load weights; mark the commit edge and the first edge that uses them. Then mark each activation's row-entry edge, every PE hop, the partial-sum update, and the VPU output edge for one fixed path. Finally mark the buffer write address and valid bit. Repeat with one bubble, one negative value, and a bank swap at a boundary.

Explain the design using five facts: what each PE stores, how operands meet, where precision changes, which resource limits bandwidth, and how completion is detected. If any answer depends on "about two cycles", replace it with an edge table.

For a new design. When returning to any RTL project, recover state ownership and one complete transaction before reading every helper module. A familiar block name does not guarantee a familiar latency, port count, or reset behavior.

15 Use the refresher in a 90-minute exercise

The exact task is unknown. Practice a complete small design, including its interface decisions and verification, rather than trying to predict one architecture. A useful time budget is 5 minutes for the contract, 10 for state/datapath sketches, 40 for RTL, 25 for testing/debugging, and 10 for explanation and cleanup. Adjust when the interviewer provides a testbench or a fixed interface.

Practice task	What to derive before coding
Enabled modulo counter	Width, terminal count, disabled behavior, wrap pulse, limit 1. Then change the limit to a non-power-of-two value.
One-entry register, then FIFO	Accepted transfers and occupancy. Add simultaneous push/pop, long stalls, depth 1, and pointer wrap. Then replace combinational storage reads with a one-cycle RAM response.
Two clients, one RAM port	Grant at most one request; advance the chosen client only on acceptance. Fixed priority is simple but can starve; round-robin adds a next-priority state bit. Route each read response using the accepted request's client ID.
RAM-fed sum or dot product	Separate request issue from response acceptance. Track outstanding work and complete only after the final result is consumed, not after the last address is issued.
Pipelined signed MAC	Product/accumulator widths, valid and metadata alignment, overflow policy. Insert a bubble and check consecutive distinct results.
Tiled two-lane datapath	Enumerate accesses per cycle, choose banks/layout, identify conflicts, then decide whether loading and computation can overlap.

Before calling a block finished:

1. State the input/output contract and the meaning of every handshake.
2. Name each register's owner and what makes it update or hold.
3. Trace reset, a normal transfer, simultaneous events, and the final item.
4. Check widths, signedness, address units, and numeric overflow.
5. Match memory port count, response latency, and collision behavior.
6. Follow valid and metadata through every pipeline boundary.
7. Run self-checking tests with a timeout and the relevant boundary cases.
8. Explain one changed assumption and the resulting architectural change.

If time is short, revisit §1, §2, §5, §7, and the project's PE trace. The highest-value habit is to say what must happen at an edge before writing the assignment.

16 Sources and runnable material

The guide uses plain technical English with short instructions and concrete examples, inspired by controlled-language writing. It does not claim formal ASD-STE100 certification. The linked primary sources are for checking a specific detail; they are not additional required reading.

- tiny-tpu source snapshot: the project map and implementation-specific observations.
- lowRISC Verilog style guide; AMD synthesis: always procedures: procedural ownership and synthesis-oriented coding.
- Intel design recommendations; Intel Quartus design recommendations: memory inference and reset limitations.
- IHP Open PDK SRAM documentation: concrete macro interfaces and implementation views.
- Microchip DDR timing parameters: examples of device timing constraints.
- Stanford AHA CGRA overview: configurable compute, memory tiles, and interconnect.
- AMD CDC guidance: asynchronous paths and crossing strategies.
- cocotb 2.0 timing model; Python runner; API reference: simulation phases and the small test workflow.

`examples/` contains eight RTL examples and the printed cocotb test. `tests/test_blocks.py` supplies independent arithmetic/queue/cycle models. `verify.py` runs the examples, including parameter boundaries, and writes `verification.json` on success. See `README.md` for setup and exact validation scope. These educational checks do not establish physical timing closure, CDC signoff, or complete project correctness.